

ProjectFlow API Reference

Comprehensive Developer Documentation & Endpoint Specification

1. Authentication & Authorization

All protected API endpoints require a valid JSON Web Token (JWT) provided in the HTTP request header:

```
Authorization: Bearer <token>
```

Unprotected public endpoints include: /auth/register, /auth/login, and /auth/explore.

Authentication Endpoints

POST /auth/register

Creates a new user account.

```
// Request Body  "username": "user1", "password": "password123"
// Response (201 Created)  "message": "User registered successfully", "recoveryKey":
"rec_8f9a2b3c4d5e"
```

POST /auth/login

Authenticates user credentials and returns a bearer token.

```
// Request Body  "username": "user1", "password": "password123"
// Response (200 OK)  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "message": "Login
successful"
```

POST /auth/explore

Generates a temporary session token for an isolated demo account.

```
// Response (200 OK)  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "message": "Demo
session initialized"
```

GET /auth/me

Fetches profile information for the currently authenticated user.

```
// Response (200 OK)  "id": 1, "username": "user1", "isDemo": false
```

POST /auth/logout

Invalidates current session/token.

```
// Response (200 OK)  "message": "Logged out successfully"
```

2. Projects API

Query Parameters for GET /projects

Parameter	Values / Format	Description
filter	all, active, overdue, cancelled, completed	Filters projects by status (Default: all).
sortOrder	asc, desc	Sorts projects by creation/due date (Default: asc).

Project Endpoints

GET /projects

Retrieves a list of projects matching specified query filters. Returns objects containing a dynamically derivedStatus.

```
// Response (200 OK) [ { "id": 1, "name": "E-Commerce Rebuild", "description": "Full redesign using React and Node", "dueDate": "2026-11-30", "cancelled": false, "techStack": ["React", "Node.js", "PostgreSQL"], "derivedStatus": "active" } ]
```

GET /projects/:id

Fetches a single project by its unique ID.

```
// Response (200 OK) { "id": 1, "name": "E-Commerce Rebuild", "description": "Full redesign using React and Node", "dueDate": "2026-11-30", "cancelled": false, "techStack": ["React", "Node.js"] }
```

POST /projects

Creates a new project item.

```
// Request Body { "name": "Mobile Application", "description": "React Native cross-platform app", "dueDate": "2026-12-15", "cancelled": false, "techStack": ["React Native", "GraphQL"] }
// Response (201 Created) { "id": 2, "message": "Project created successfully" }
```

PUT /projects/:id

Updates an existing project. All payload fields are optional.

```
// Response (200 OK) { "message": "Project updated successfully" }
```

DELETE /projects/:id

Deletes a project by ID.

```
// Response (200 OK) { "message": "Project deleted successfully" }
```

3. Tasks API

Query Parameters for GET /tasks

Parameter	Values / Defaults	Description
projectId	Number (optional)	Filters tasks associated with a specific project.
status	all, open, completed, overdue	Task completion status (Default: all).
priority	all, high, medium, low	Task priority level (Default: all).
sortOrder	asc, desc	Order by due date or priority (Default: asc).
page	Integer (Default: 1)	Page number for paginated results.
limit	Integer (Default: 10)	Number of items per page.

Task Endpoints

GET /tasks

Retrieves a paginated list of tasks according to filters.

```
// Response (200 OK) "data": [ "id": 1, "name": "Design DB Schema", "projectId": 1, "status":  
"open", "priority": "high", "dueDate": "2026-09-01", "description": "Draft relational models  
for users and orders" ], "pagination": "currentPage": 1, "limit": 10, "totalItems": 5,  
"totalPages": 1
```

GET /tasks/:id

Fetches a single task by ID.

POST /tasks

Creates a new task.

```
// Request Body "name": "Implement Auth Middleware", "projectId": 1, "status": "open",  
"priority": "high", "dueDate": "2026-09-10", "description": "JWT verify middleware for API  
routes"  
// Response (201 Created) "id": 2, "message": "Task created successfully"
```

PUT /tasks/:id

Updates task details. Fields are optional.

DELETE /tasks/:id

Removes a task by ID.

4. Notes API

GET /tasks/:taskId/notes

Retrieves all notes linked to a specific task ID.

```
// Response (200 OK) [ "id": 101, "taskId": 1, "content": "Remember to index foreign key columns  
for fast joins.", "createdAt": "2026-08-15T10:00:00Z" ]
```

GET /notes/:id

Retrieves a specific note by ID.

POST /notes

Attaches a new note to a task.

```
// Request Body "content": "Updated database diagram with user role tables.", "taskId": 1  
// Response (201 Created) "id": 102, "message": "Note added successfully"
```

PUT /notes/:id

Updates the content of an existing note.

```
// Request Body  "content": "Revised note content text..."
```

DELETE /notes/:id

Deletes a note by ID.

5. Attachments API

Upload Constraints

- **Content-Type:** multipart/form-data
- **Allowed File Extensions:** .png, .jpg, .jpeg, .svg, .pdf, .md, .txt

Attachment Endpoints

GET /projects/:projectId/attachments

Fetches metadata list for all attachments linked to a project.

```
// Response (200 OK) [  {"id": 501, "projectId": 1, "fileName": "architecture_diagram.png",  
  "mimeType": "image/png", "sizeBytes": 204850  } ]
```

GET /attachments/:id

Retrieves metadata for a specific attachment.

POST /attachments

Uploads a file to a project using multipart/form-data.

```
// Form Fields file: [binary data] projectId: 1  
// Response (201 Created)  {"id": 502, "message": "File uploaded successfully"}
```

GET /attachments/:id/download

Downloads the raw file stream for the specified attachment ID.

DELETE /attachments/:id

Deletes an attachment from storage and removes metadata.